

EMPLOYEE HIERARCHY MANAGEMENT SYSTEM USING C LANGUAGE

MAHESH KUMAR REDDY M
Department of Computer
Science
College: SAI UNIVERSITY

Gowtham P
Department of Computer
Science
College: SAI UNIVERSITY

1. Introduction

Organizations today handle large volumes of employee data, making manual management inefficient and error-prone. Hierarchical data representation is essential to maintain reporting relationships, organizational structure, and workflow management. Tree data structures provide a natural representation of such hierarchical relationships, where each node represents an entity and edges represent relationships.

Unlike linear data structures, trees allow efficient traversal and dynamic insertion and deletion of nodes. This makes them highly suitable for applications such as file systems, database indexing, and organizational management systems. The Employee Hierarchy Management System leverages these properties to provide an efficient solution for managing employee relationships.

2. Problem Statement

In large-scale organizations, managing employee relationships becomes increasingly complex due to multiple levels of hierarchy. Traditional storage methods such as arrays and linked lists are not suitable for representing hierarchical relationships effectively, as they fail to capture parent-child dependencies.

Several challenges arise in such systems. Maintaining consistency when employees are added or removed becomes difficult, especially when multiple levels of reporting exist. Efficiently searching for employees in large datasets is another major concern. Additionally, managing subordinate relationships during deletion operations is complex, as removing one employee may affect an entire branch of the hierarchy.

This project addresses these challenges by using a tree-based data structure, ensuring that all hierarchical dependencies are maintained correctly while supporting efficient operations.

3. System Design

3.1 Node Structure Design

In this system, each employee is represented as a node using a structure in the C programming language. The node contains an integer field for the Employee ID, which acts as a unique identifier. It also includes character arrays to store the employee's name and designation.

To represent hierarchical relationships, each node contains two pointers: one pointing to the first child and another pointing to the next sibling. This design allows

efficient representation of a general tree structure while minimizing memory usage compared to other methods.

3.2 Child–Sibling Representation Advantage

The child–sibling representation converts a general tree into a binary-like structure, making implementation and traversal simpler. Instead of storing multiple child pointers, each node only needs two pointers, reducing complexity.

This model avoids the need for dynamic arrays to manage children and simplifies recursive operations such as traversal, insertion, and deletion. It is particularly useful in representing hierarchical data efficiently.

4. Data Structure Analysis

The system uses a tree as a non-linear data structure to

represent employee hierarchy. In this structure, the root node represents the CEO, intermediate nodes represent managers, and leaf nodes represent employees who do not have subordinates.

Each node can have multiple children but only one parent, ensuring a clear reporting structure. The tree is acyclic, meaning there are no loops, which maintains logical consistency in hierarchy representation.

To traverse the tree, Depth First Search (DFS) is used. Preorder traversal is specifically applied to display the hierarchy in a structured and readable format.

5. Algorithm Analysis

The insertion operation involves first searching for the manager node under whom the new employee will be added. This ensures that the hierarchical structure

remains intact and logically consistent.

For searching operations, the system uses Depth First Search (DFS), which is efficient for tree traversal as it explores each branch completely before moving to the next. This approach ensures that all nodes are visited when necessary.

The deletion operation is more complex, as it involves removing a node from its parent and recursively deleting all its child nodes. Proper memory deallocation is performed using free() to avoid memory leaks. This ensures efficient memory management within the system.

6. Implementation

The system is implemented using modular programming principles, where each operation is handled by a separate function. This improves code readability,

maintainability, and scalability.

The implementation uses structures (struct) to define employee nodes and dynamic memory allocation through malloc(). Memory is properly managed using free() to avoid leaks. Recursive functions are used for traversal operations, making the implementation efficient and clean.

A menu-driven interface is provided for user interaction, allowing users to perform operations such as insertion, deletion, search, and display easily. Error handling mechanisms are also included to manage invalid inputs and null pointer scenarios.

7. Results and Discussion

The system was tested with multiple employee entries to evaluate its performance and correctness. The hierarchy was successfully created

and displayed in a structured and readable format.

The system effectively maintains parent-child relationships, ensuring that the hierarchy remains consistent after every operation. The search operation successfully locates employees within the structure, while the update operation modifies employee details without affecting the hierarchy.

The deletion operation correctly removes an employee along with all its subordinates, ensuring structural integrity. In terms of performance, the system works efficiently for small to medium-sized datasets, with execution time increasing linearly as the number of nodes grows.

However, some limitations were observed. In the worst case, searching requires full traversal of the tree, leading to increased time complexity. Additionally, the system does

not include persistent storage, meaning all data is lost once the program execution ends.

8. Applications

This system has wide applications in various domains. It can be used in corporate employee management systems to maintain organizational structures. It is also useful for generating organizational charts and representing university faculty hierarchies.

Additionally, the system can be applied in file system directory structures and project team management tools. It serves as a strong foundation for developing advanced systems such as Enterprise Resource Planning (ERP) and Human Resource Management Systems (HRMS).

9. Advantages

The system provides an efficient way to represent hierarchical relationships using a tree structure. It is flexible and supports dynamic insertion and deletion of nodes. The use of recursion simplifies implementation, while pointers ensure memory efficiency.

The structure supports real-time updates and maintains consistency throughout operations, making it suitable for practical applications.

10. Limitations

Despite its advantages, the system has certain limitations. Most operations have a time complexity of $O(n)$, which may affect performance for very large datasets. The system does not include graphical visualization, making it less user-friendly.

Scalability is limited for extremely large organizations, and the system relies on manual input, which may introduce errors.

11. Future Scope

The system can be significantly enhanced with future improvements. Integration with databases can provide persistent storage, allowing data to be retained even after program execution.

Developing a graphical user interface (GUI) would improve usability and visualization. Advanced data structures such as AVL trees or B-Trees can be implemented to improve efficiency.

Additionally, deploying the system as a web-based application would enable real-time access, and incorporating role-based authentication would enhance security.

12. Conclusion

This project demonstrates how tree data structures can effectively model real-world hierarchical systems. The use of child–sibling representation simplifies implementation while maintaining flexibility and efficiency.

The system provides a strong foundation for understanding advanced data structures and can be extended into real-world applications with additional features and enhancements.

13. References

[1] T. H. Cormen, *Introduction to Algorithms*, MIT Press, 2009.

[2] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*.

[3] Horowitz and Sahni, *Fundamentals of Data Structures*.

[4] GeeksforGeeks, “Tree Data Structure,” Online Resource.

[5] IEEE Standards Association, *IEEE Paper Format Guidelines*